

Relatório de evolução - Sprint 03

ChargeAssistente · GoodWe EV ChargeOps

1. Resumo da evolução

Nas Sprints 1 e 2, o ChargeAssistente usava Streamlit, SDK direto do Ollama, Llama 3 8B e histórico manual limitado a 20 mensagens. A Sprint 03 reconstruiu o núcleo com LangChain LCEL (prompt | llm | parser), memória por sessão limitada a 1.800 tokens, schema Pydantic v2, XML tagging, medição com tiktoken e guardrails determinísticos.

2. Refatoração e decisões técnicas

O ChatPromptTemplate reúne prompt, histórico e entrada. O ChatOllama permite trocar o modelo sem alterar a chain e o StrOutputParser entrega texto à interface. RunnableWithMessageHistory separa sessões e um adaptador usa ConversationTokenBufferMemory. Esses componentes foram mantidos por serem os exigidos no enunciado, apesar dos avisos de depreciação na versão atual do LangChain.

O schema ConsultaRecarga normaliza o identificador, aceita decimal com vírgula e valida estado, potência, energia e valor. O prompt v3 usa tags XML e proíbe inventar recursos, disponibilidade e especificações. Guardrails externos tratam injection, escopo, segurança elétrica e aconselhamento jurídico ou financeiro antes da chamada ao modelo.

3. Comparativo antes/depois

As duas versões foram executadas no mesmo Mac, com qwen3:4b-instruct, temperature 0,2, top_p 0,9, num_predict 512 e o mesmo conjunto de 11 casos. A qualidade exige decisão correta e critérios mínimos de conteúdo.

Métrica	Sprints 1/2 - manual	Sprint 03 - LCEL
Qualidade no eval	63,64% (7/11)	100% (11/11)
Tokens médios de entrada	13,27	13,27
Tokens médios de saída	118,09	161,55
Latência média	22,38 s	19,54 s
Acurácia structured output	Não disponível	100% (3/3)

O ganho principal foi de 36,36 pontos percentuais na qualidade e 12,69% de redução na latência. O aumento dos tokens de saída decorre de respostas mais completas e orientações seguras. O prompt v3 tem 695 tokens, contra 1.842 na v2.

4. Problemas encontrados e soluções

- 1 O modelo obrigatório gpt-oss:120b ocupa cerca de 65 GB e não cabe na máquina de desenvolvimento. A chain usa esse nome como padrão e aceita OLLAMA_MODEL/OLLAMA_BASE_URL; os testes reproduzíveis foram realizados com Qwen local e o teste final de 120B deve usar Ollama remoto.
- 2 O primeiro eval marcava toda resposta liberada como correta, mesmo quando inventava um aplicativo ou recomendava lavar conectores. Foram adicionados critérios de conteúdo por caso e proibições explícitas no prompt.
- 3 A execução direta do arquivo causava ModuleNotFoundError. O projeto passou a documentar a execução como módulo: python -m evals.run_evals.